MySQL – Database and Table Fundamentals

1. Introduction to MySQL

MySQL is a Relational Database Management System (RDBMS) that stores data in tables consisting of rows and columns.

- **Database:** A collection of related tables.
- **Table:** Stores data in rows and columns.
- **Row (Record):** A single entry in a table.
- **Column (Field):** An attribute that defines the type of data stored.

MySQL uses SQL (Structured Query Language) to create, retrieve, update, and delete data.

---

2. Basic MySQL Commands

Clear the Command Prompt

```
system cls;
```

Display Available Databases

```
SHOW DATABASES;
```

Select a Database

```
USE world;
```

The selected database becomes the default database for subsequent queries.

Display Tables

```
SHOW TABLES;
```

Describe a Table

```
DESCRIBE countrylanguage;
```

Alternative:

```
DESC countrylanguage;
```

Displays:

- Field: Column name
- Type: Data type
- Null: Whether NULL values are allowed
- Key: Key information (PRI, UNI, MUL)
- Default: Default value
- Extra: Additional properties such as AUTO_INCREMENT

Display All Records

```
SELECT * FROM country;
```

Display Specific Columns

```
SELECT name, population FROM country;
```

Syntax:

```
SELECT column_name FROM table_name;
```

---

3. Creating a Database

```
CREATE DATABASE mango;
```

To avoid an error if the database already exists:

```
CREATE DATABASE IF NOT EXISTS mango;
```

Select the database:

```
USE mango;
```

To delete a database:

```
DROP DATABASE mango;
```

Interview point: `DROP DATABASE` permanently removes the database and its tables.

---

4. MySQL Data Types

A data type defines what kind of values a column can store.

Choosing appropriate data types helps optimize storage, performance, and data integrity.

A. String Data Types

Data Type	Description
CHAR(n)	Fixed-length character string
VARCHAR(n)	Variable-length character string
TEXT	Stores large text values
BLOB	Stores binary data
ENUM	Allows one value from a predefined list
SET	Allows multiple values from a predefined list

CHAR vs VARCHAR

- `CHAR(10)` : Fixed-length field. Suitable for values with predictable lengths.
- `VARCHAR(50)` : Variable-length field. Suitable for names, addresses, etc.

Example:

```
name CHAR(25),
address VARCHAR(100)
```

B. Numeric Data Types

Data Type	Description
TINYINT	Very small integer
SMALLINT	Small integer
INT	Standard integer
BIGINT	Large integer
DECIMAL(p,s)	Exact decimal values
FLOAT	Approximate floating-point values
DOUBLE	Higher-precision approximate floating-point values
BOOLEAN / BOOL	Boolean-like values (alias of TINYINT(1))

Important integer ranges (signed):

Type	Range
TINYINT	-128 to 127
SMALLINT	-32,768 to 32,767
INT	-2,147,483,648 to 2,147,483,647
BIGINT	-2^{63} to $2^{63}-1$

Adding `UNSIGNED` removes the negative range.

Example:

```
age INT UNSIGNED
```

Understanding DECIMAL(p,s)

- `p` = Total number of digits.
- `s` = Number of digits after the decimal point.

Example:

```
salary DECIMAL(7,2)
```

Maximum positive value:

```
99999.99
```

Another example:

```
price DECIMAL(5,2)
```

Maximum positive value:

```
999.99
```

Interview point: Use DECIMAL for financial values because it stores exact decimal values. FLOAT and DOUBLE use approximate floating-point representation.

C. Date and Time Data Types

Data Type	Description	Format
DATE	Date only	YYYY-MM-DD
TIME	Time only	HH:MM:SS
DATETIME	Date and time	YYYY-MM-DD HH:MM:SS
TIMESTAMP	Date and time, stored with UTC conversion	YYYY-MM-DD HH:MM:SS
YEAR	Year value	YYYY

Example:

```
dob DATE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

Interview point:

- DATE is suitable for birthdays.
- DATETIME is suitable for storing date and time without automatic timezone conversion.
- TIMESTAMP is useful for recording events with timezone-aware UTC conversion.

---

5. Creating Tables

After selecting a database, create a table using `CREATE TABLE` .

Example:

```
CREATE TABLE students (
    rollno INT,
    name CHAR(25),
    city CHAR(20),
    address VARCHAR(50),
    pincode CHAR(6),
    mobileno DECIMAL(10,0),
    dob DATE,
    attendance BOOLEAN
);
```

Check the table:

```
SHOW TABLES;
DESC students;
```

Understanding the Table Structure

Column	Data Type	Purpose
rollno	INT	Student roll number
name	CHAR(25)	Student name
city	CHAR(20)	City

Column	Data Type	Purpose
address	VARCHAR(50)	Address
pincode	CHAR(6)	Postal code
mobileno	DECIMAL(10,0)	Mobile number
dob	DATE	Date of birth
attendance	BOOLEAN	Attendance status

Important: Mobile numbers and PIN codes are generally better stored as `VARCHAR` or `CHAR`, not numeric types. They are identifiers, not values used for mathematical calculations. This also preserves leading zeros.

For example:

```
mobileno VARCHAR(15),
pincode CHAR(6)
```

---

6. Constraints in MySQL

Constraints are rules applied to columns to maintain data accuracy and integrity.

Constraint	Purpose
NOT NULL	Prevents NULL values
UNIQUE	Prevents duplicate values
PRIMARY KEY	Uniquely identifies each record
FOREIGN KEY	Establishes relationships between tables
DEFAULT	Assigns a default value
CHECK	Enforces a condition on values
AUTO_INCREMENT	Automatically generates numeric values

Example

```
CREATE TABLE students (
    rollno INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    city VARCHAR(30),
    attendance BOOLEAN DEFAULT FALSE
);
```

Primary Key

A primary key uniquely identifies each row in a table.

Properties:

- Must contain unique values.
- Cannot contain NULL.
- A table can have only one primary key.
- A primary key can consist of multiple columns (composite primary key).

Example:

```
rollno INT PRIMARY KEY
```

AUTO_INCREMENT

Automatically generates the next numeric value when a new record is inserted.

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    city VARCHAR(50)
);
```

Insert records without specifying the ID:

```
INSERT INTO students (name, city)
VALUES ('Arya', 'Nashik');
```

MySQL automatically generates the ID.

Interview point: AUTO_INCREMENT is commonly used with an integer primary key.

---

7. Inserting Records

Use the `INSERT INTO` statement to add records.

Method 1: Insert values into all columns

```
INSERT INTO students
VALUES (
    1,
    'Arya',
    'Nashik',
    '15 Chandramangal Colony',
    '422009',
    '9970643818',
    '2026-11-01',
    TRUE
);
```

The values must follow the table's column order.

Method 2: Specify column names

```
INSERT INTO students
(name, city, address, pincode, mobileno, dob, attendance)
VALUES (
    'Neeraj',
    'Pune',
    'New Pune',
    '422002',
    '9870643818',
    '2026-11-02',
    TRUE
);
```

This method is recommended because:

- Column order does not need to match the table definition.
- AUTO_INCREMENT columns can be omitted.
- It makes queries easier to understand.

Insert Multiple Records

```
INSERT INTO students (id, name, city)
VALUES
(1, 'Arya', 'Nashik'),
(2, 'Neeraj', 'Pune'),
(3, 'Rahul', 'Mumbai');
```

---

8. Retrieving Records – SELECT

Retrieve All Columns

```
SELECT * FROM students;
```

Retrieve Specific Columns

```
SELECT name, city FROM students;
```

WHERE Clause

Used to filter records based on a condition.

```
SELECT * FROM students
WHERE city = 'Pune';
```

Other operators:

Operator	Purpose
=	Equal
!= or <>	Not equal
>	Greater than
<	Less than

Operator	Purpose
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Within a range
IN	Matches values in a list
LIKE	Pattern matching
IS NULL	Checks NULL values

Examples:

```
SELECT * FROM students WHERE rollno > 1;

SELECT * FROM students WHERE city IN ('Pune', 'Nashik');

SELECT * FROM students WHERE name LIKE 'A%';

SELECT * FROM students WHERE city IS NULL;
```

LIKE wildcards:

- `%` – Matches zero or more characters.
- `_` – Matches exactly one character.

---

9. Updating Records – UPDATE

The `UPDATE` statement modifies existing records.

Syntax:

```
UPDATE table_name
SET column_name = value
WHERE condition;
```

Example:

```
UPDATE students
SET attendance = TRUE
WHERE name = 'Neeraj';
```

Update multiple columns:

```
UPDATE students
SET city = 'Mumbai', attendance = FALSE
WHERE rollno = 2;
```

Important: Always use the WHERE clause when you intend to update specific records.

Without WHERE:

```
UPDATE students
SET attendance = TRUE;
```

Every row in the table will be updated.

---

10. Deleting Records – DELETE

The `DELETE` statement removes records from a table.

Delete a particular record:

```
DELETE FROM students
WHERE rollno = 2;
```

Delete all records:

```
DELETE FROM students;
```

The table structure remains intact.

DELETE vs TRUNCATE vs DROP

Command	Function
DELETE	Removes selected or all rows
TRUNCATE	Removes all rows and resets AUTO_INCREMENT
DROP	Removes the entire table, including its structure

Example:

```
TRUNCATE TABLE students;

DROP TABLE students;
```

---

11. Important Interview Questions

1. What is the difference between CHAR and VARCHAR?

CHAR stores fixed-length strings, whereas VARCHAR stores variable-length strings.

2. What is the difference between PRIMARY KEY and UNIQUE?

Both enforce uniqueness. A primary key cannot contain NULL and there can only be one primary key per table. A table can have multiple UNIQUE constraints.

3. What is AUTO_INCREMENT?

It automatically generates sequential numeric values when records are inserted.

4. What is the difference between DELETE and TRUNCATE?

DELETE removes records and supports WHERE. TRUNCATE removes all records without WHERE and resets the AUTO_INCREMENT counter.

5. What is the difference between DROP and DELETE?

DELETE removes records but preserves the table. DROP removes the entire table structure and its data.

6. What is NULL?

NULL represents a missing or unknown value. It is different from zero, an empty string, or FALSE.

To check NULL:

```
SELECT * FROM students WHERE city IS NULL;
```

7. What is the difference between WHERE and HAVING?

WHERE filters individual rows before grouping. HAVING filters groups after GROUP BY.

8. Why use DECIMAL instead of FLOAT for money?

DECIMAL provides exact decimal representation, while FLOAT uses approximate binary floating-point representation.

---

12. Quick Revision – Essential Commands

Task	Command
Show databases	`SHOW DATABASES;`
Create database	`CREATE DATABASE mango;`
Select database	`USE mango;`
Show tables	`SHOW TABLES;`
Describe table	`DESC students;`
Create table	`CREATE TABLE ...;`
Insert record	`INSERT INTO ... VALUES ...;`
Display records	`SELECT * FROM students;`
Filter records	`SELECT * FROM students WHERE ...;`
Update records	`UPDATE students SET ... WHERE ...;`
Delete records	`DELETE FROM students WHERE ...;`
Delete all records	`TRUNCATE TABLE students;`
Delete table	`DROP TABLE students;`
Delete database	`DROP DATABASE mango;`

Remember: SQL commands are generally case-insensitive, but string values depend on the column's collation. Always terminate SQL statements with a semicolon (`;`).